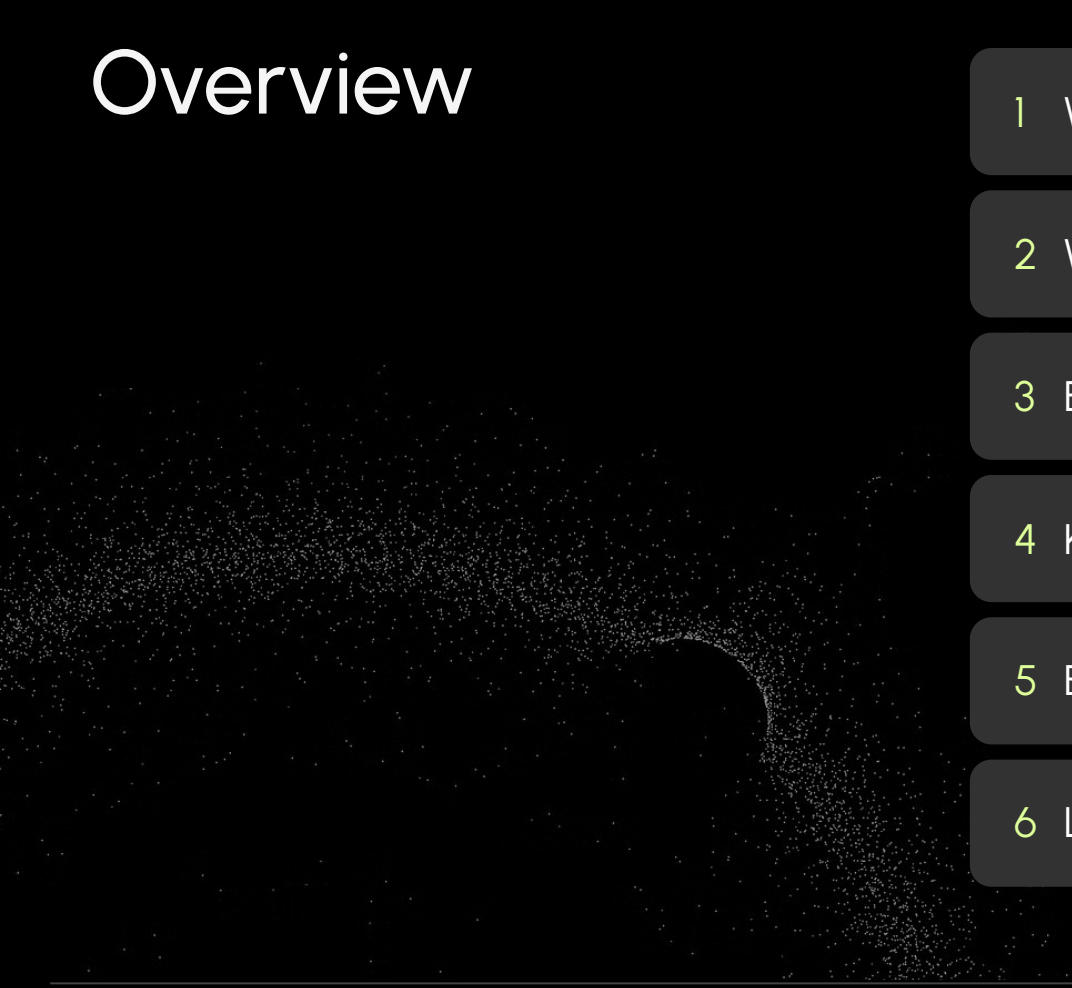
Intro to Windows Driver Vulnerability Analysis

Laurel

Overview



1 Windows Internals

2 Windows Drivers

3 Exploitation Concepts

4 Kernel Exploitation

5 Exercise - Driver Analysis

6 Learning More

About Laurel

Laurel is a software developer with an interest in exploitation and reverse engineering. She has experience with Windows kernel driver development, kernel-mode debugging, reverse engineering, and vulnerability analysis using tools including WinDbg and IDA.

Discord / GitHub: @lw8192

Environment Setup

IDA Free with activated license

Vulnerable driver > Click "Download vulnerable sample"

(downloaded as a .bin file - don't rename it!)

Windows Architecture

What is a kernel? Core of the OS. Windows - hybrid kernel (combo of microkernel and monolithic).

Microkernel - bare minimum of OS services. Monolithic - core OS services in 1 address space.

Kernel and Windows executive are located in **ntoskrnl.exe**.

Hardware Abstraction Layer (HAL) - hal.dll. Between hardware and the rest of the OS. Low level hardware interaction APIs.

Win32 API: documented, user-mode, in kernel32.dll and others.

Native API: mostly undocumented, in ntdll.dll.

Kernel vs User Mode

Modern OS: privilege rings. Restricts access to system resources.

Ring 0 vs Ring 3: separate user and kernel code for security.

Ring 0 (kernel-mode): Windows executive, drivers, HAL. Drivers run in the same address space as the kernel.

Ring 3 (user-mode): user mode processes, some services, applications and system owned processes, environment subsystem.

Normal user mode application: can't directly interact with the kernel or modify physical hardware. When a system call is made - processor switches from user to kernel mode.

Kernel vs User Mode

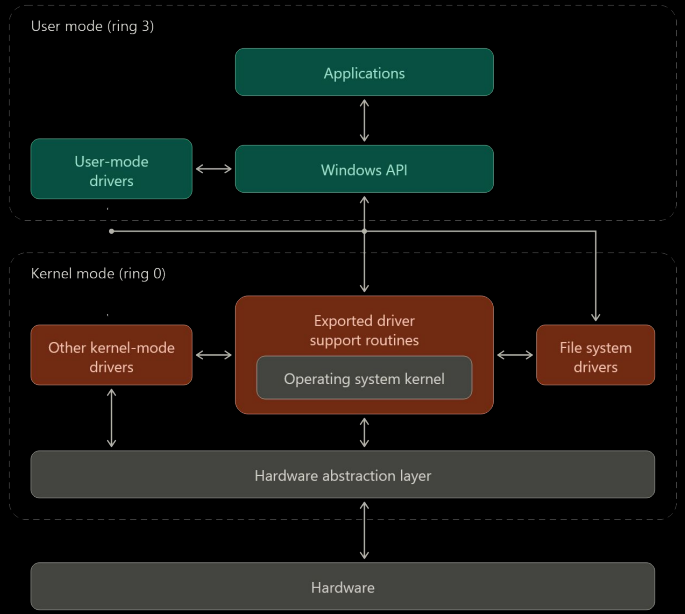


Image modified from: Microsoft Docs (User and Kernel mode)

User / kernel mode crossing

System call: operation exposed by the OS. An application can invoke a system call to get the OS to perform services with privileged access.

OS needs to context switch from a user mode process to kernel.

IOCTL(Input/Output Control): a control code used to request device/driver-specific operations, defined by the system or in the driver.

Memory Management

Virtual addressing: each user-mode process gets a "private" virtual address space which gets translated to a physical address.

Each process has: executable code, open handles to system objects, a security context, a PID, environment variables, a priority class, minimum and maximum working set sizes, and 1+ threads.

Kernel-mode code: shares virtual system space.

Paged vs non-paged memory pool

Memory pools: memory used by the kernel / drivers. Kernel objects that allow chunks of memory to be allocated from a designated memory region, paged / non-paged memory.

Paged pool: virtual memory that can be paged (read/written) to the page file on disk, c:\pagefile.sys. **Recommended pool for drivers to use.**

Non-paged pool: can't be paged out, always lives in RAM. Needed when page faults can't be tolerated.

Kernel-Mode Data Structures

EPROCESS: executive process, for the kernel to maintain info on each process. **Contains the access token for the process (a common target for attackers).**

KPROCESS: kernel process object, in EPROCESS struct.

ETHREAD: executive level representation of a thread.

KTHREAD: kernel level representation of a thread. In ETHREAD.

Windows Object Manager: manages instances of these objects.

You can find undocumented structs using the Vergilius project, ntdocs or by dumping structs in WinDbg.

Note: offsets and layouts of these structs vary across builds!

Windows Drivers

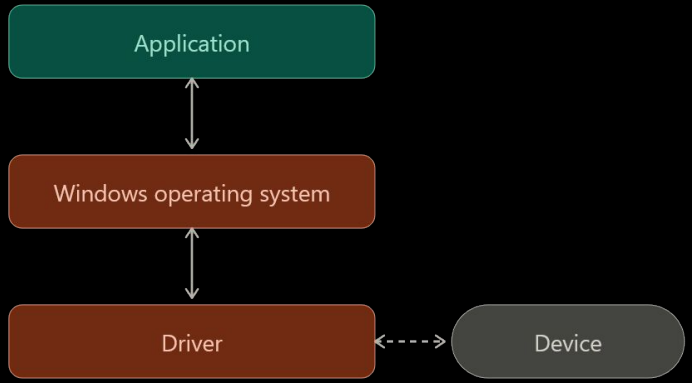


Image from: Microsoft Docs (What is a Driver)

Driver Overview

Driver: for the OS and a device to communicate. Written in C / C++.

Kernel driver: .sys file, must be loaded into the OS.

WDM = Windows Driver Model. Legacy but still used today.

WDF = Windows Driver Framework. Most widely used today, encapsulates WDM. Made up by the KMDF and UMDF (Kernel-Mode Driver Framework, User-Mode Driver Framework).

From a vulnerability perspective: in a WDM driver the dev must handle more I/O functionality (like raw IRPs, memory management and synchronization) which leads to more vulnerabilities! The WDF provides abstractions / code to handle more of this.

Microsoft Docs - What is a Driver?

Driver Standard Routines

DriverEntry: driver initialization, like a main() function. This gets called when the driver is loaded into memory. Sets up device name, symbolic link and registers IRP dispatch routines.
Dispatch routines: handle requests made to the driver. Registering a routine - means setting it as a callback function (function to be called when a certain action occurs).
DriverUnload: unloads the driver, deletes device objects and symbolic links.
Driver Standard Routines

Windows Driver Variables

Variables: what you see in C code.

DriverObject: variable in the driver's code that represents image of a loaded driver, used in device creation and dispatch routine initialization.

DeviceObject: let processes interact with the driver, kernel representation of a device that receives I/O requests.

Symbolic link: user-visible name that allows an application to open a handle to the device and send messages to it (via IRPs).

WDM vs WDF Drivers

WDF - uses wrapper functions for functions WDM drivers use.

You will see: WdfDeviceCreate(), WdfDeviceCreateSymbolicLink()

WDM - lower level version of these functions. The kind of drivers we want to look at for vulnerabilities!

You will see: IoCreateDevice(), IoCreateSymbolicLink()

I/O System

I/O system: manages flow of requests to drivers.

IRP (I/O Request Packet): kernel data structure with request info.

I/O manager: creates and sends IRPs to a driver.

Device stack: IRP is sent through multiple levels to reach its destination, ordered list of device objects. Sent through stack then back once the request is filled. A dispatch routine is called to fulfill an IRP.

Device Nodes and Device Stacks

IRP Request Flow

1. Manager creates I/O request (IRP).
2. Request is sent to the device stack.
3. IRP goes through the device stack - devices / driver perform operations or pass the IRP on.
4. IRP reaches the target driver - dispatch routine is called.
5. Once completed - IoCompleteRequest() is called, IRP is passed back through the device stack.
6. IRP comes back to the IRP manager.

Example I/O Request

IOCTLs

An IRP is sent to trigger an IOCTL and call a certain function in the driver.

An IOCTL is a 32-bit control code identifying a device or driver-specific operation.

Defined by the driver. Passed to DeviceIoControl().

Vulnerabilities in kernel-mode drivers often come from IOCTLs that allow actions they should not.

A driver routine from user space that can be called through DeviceIoControl() - might be able to find a vulnerability you can take advantage of.

Defining I/O Control Codes (bit-wise layout)

WDM DriverEntry Pseudocode

```
NTSTATUS __stdcall DriverEntry(_DRIVER_OBJECT * Driver, ...){
    HANDLE device_handle;
    //initialize DeviceName and SymbolicLink
    device_handle = IoCreateDevice(DriverObject, DeviceExtensionSize,
&DeviceName, DeviceType, DeviceCharacteristics, Exclusive, &DeviceObject);
    IoCreateSymbolicLink(&SymbolicLinkName, &DeviceName);
    //MajorFunction table: defines what functions are called for each code.
    DriverObject->MajorFunction[IRP_MJ_CREATE] = driver_create;
    DriverObject->MajorFunction[IRP_MJ_CLOSE] = driver_close;
    DriverObject->MajorFunction[IRP_MJ_DEVICE_CONTROL] = IOCTL_handler;
    //a user-mode process will use DeviceIoControl() with an IRP code to
    call each specific function
}
```

WDM IOCTL handler Pseudocode

```
NTSTATUS IOCTL_handler(PDEVICE_OBJECT DeviceObject, PIRP Irp){
    //parse out IOCTL code from passed args
    //usually a switch case or if/else statements against the IOCTL code
    //usually IOCTL codes will be defined in a header
    //when reversing you will see IOCTLs as hex or decimal numbers
    switch(ioctl_code){
        case DEFINED_IOCTL_CODE:
            //do things for this IOCTL
            //possibly vulnerable functions here???
    }
}
```

Example analysis - HEVD

```
__int64 __fastcall DriverEntry(_DRIVER_OBJECT *DriverObject, _UNICODE_STRING *RegistryPath)
{
  NTSTATUS v3; // edi
  _UNICODE_STRING DosDeviceName; // [rsp+40h] [rbp-20h] BYREF
  _UNICODE_STRING DeviceName; // [rsp+50h] [rbp-10h] BYREF
  _DEVICE_OBJECT *DeviceObject; // [rsp+80h] [rbp+20h] BYREF

  DeviceObject = nullptr;
  *(_QWORD *)&DosDeviceName.Length = 0;
  DosDeviceName.Buffer = nullptr;
  RtlInitUnicodeString(DestinationString: &DeviceName, SourceString: L"\\Device\\HackSysExtremeVulnerableDriver");
  RtlInitUnicodeString(DestinationString: &DosDeviceName, SourceString: L"\\DosDevices\\HackSysExtremeVulnerableDriver");
  v3 = IoCreateDevice(
         DriverObject,
         DeviceExtensionSize: 0,
         &DeviceName,
         DeviceType: 0x22u,
         DeviceCharacteristics: 0x100u,
         Exclusive: 0,
         &DeviceObject);
  if ( v3 >= 0 )
  {
    memset64(DriverObject->MajorFunction, (unsigned __int64)IrpNotImplementedHandler, 0x1Cu);
    DriverObject->MajorFunction[0] = IrpCreateCloseHandler;
    DriverObject->MajorFunction[2] = IrpCreateCloseHandler;
    DriverObject->MajorFunction[14] = IrpDeviceIoCtlHandler;
    DriverObject->DriverUnload = DriverUnloadHandler;
    DeviceObject->Flags |= 0x10u;
    DeviceObject->Flags &= ~0x80u;
    v3 = IoCreateSymbolicLink(SymbolicLinkName: &DosDeviceName, &DeviceName);
```

device name

creating the device object - WDM driver

MajorFunction table

IOCTL handler

creating the symbolic link

Example - HEVD DriverEntry assembly

```
; __int64 __fastcall DriverEntry(_DRIVER_OBJECT *DriverObject, _UNICODE_STRING *RegistryPath)
DriverEntry proc near

DeviceCharacteristics= dword ptr -40h
Exclusive= byte ptr -38h
var_30= qword ptr -30h
DosDeviceName= _UNICODE_STRING ptr -20h
DeviceName= _UNICODE_STRING ptr -10h
arg_0= qword ptr  10h
arg_8= qword ptr  18h
DeviceObject= qword ptr  20h

DriverObject = rcx
RegistryPath = rdx
mov     [rsp-8+arg_0], rbx
mov     [rsp-8+arg_8], rdi
push    rbp
mov     rbp, rsp
sub     rsp, 60h
and     [rbp+DeviceObject], 0
lea     RegistryPath, aDeviceHacksyse ; "\\Device\\HackSysExtremeVulnerableDrive"...
xor     eax, eax
mov     rbx, DriverObject
lea     DriverObject, [rbp+DeviceName] ; DestinationString
mov     qword ptr [rbp+DosDeviceName.Length], rax
mov     [rbp+DosDeviceName.Buffer], rax
call    cs:__imp_RtlInitUnicodeString ; ntoskrnl
lea     RegistryPath, aDosdevicesHack ; "\\DosDevices\\HackSysExtremeVulnerableD"...
lea     DriverObject, [rbp+DosDeviceName] ; DestinationString
call    cs:__imp_RtlInitUnicodeString ; ntoskrnl
lea     rax, [rbp+DeviceObject]
mov     r9d, 22h ; '"'  ; DeviceType
mov     [rsp+60h+var_30], rax ; DeviceObject
lea     r8, [rbp+DeviceName] ; DeviceName
mov     [rsp+60h+Exclusive], 0 ; Exclusive
xor     edx, edx        ; DeviceExtensionSize
mov     DriverObject, rbx ; DriverObject
mov     [rsp+60h+DeviceCharacteristics], 100h ; DeviceCharacteristics
call    cs:__imp_IoCreateDevice ; ntoskrnl
```

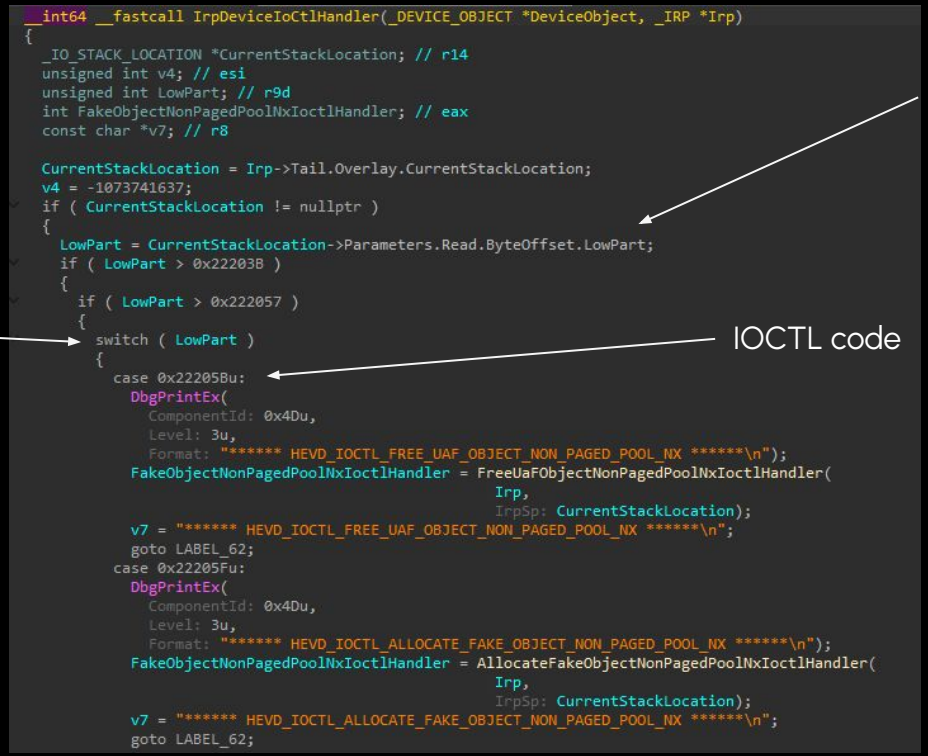
call to create device

Example - HEVD IOCTL handler

Extracting the IOCTL

switch on the IOCTL

IOCTL code



```
__int64 __fastcall IrpDeviceIoCtlHandler(_DEVICE_OBJECT *DeviceObject, _IRP *Irp)
{
  _IO_STACK_LOCATION *CurrentStackLocation; // r14
  unsigned int v4; // esi
  unsigned int LowPart; // r9d
  int FakeObjectNonPagedPoolNxIoctlHandler; // eax
  const char *v7; // r8

  CurrentStackLocation = Irp->Tail.Overlay.CurrentStackLocation;
  v4 = -1073741637;
  if ( CurrentStackLocation != nullptr )
  {
    LowPart = CurrentStackLocation->Parameters.Read.ByteOffset.LowPart;
    if ( LowPart > 0x22203B )
    {
      if ( LowPart > 0x222057 )
      {
        switch ( LowPart )
        {
          case 0x22205Bu:
            DbgPrintEx(
              ComponentId: 0x4Du,
              Level: 3u,
              Format: "****** HEVD_IOCTL_FREE_UAF_OBJECT_NON_PAGED_POOL_NX ******\n");
            FakeObjectNonPagedPoolNxIoctlHandler = FreeUaFObjectNonPagedPoolNxIoctlHandler(
                                                     Irp,
                                                     IrpSp: CurrentStackLocation);
            v7 = "****** HEVD_IOCTL_FREE_UAF_OBJECT_NON_PAGED_POOL_NX ******\n";
            goto LABEL_62;
          case 0x22205Fu:
            DbgPrintEx(
              ComponentId: 0x4Du,
              Level: 3u,
              Format: "****** HEVD_IOCTL_ALLOCATE_FAKE_OBJECT_NON_PAGED_POOL_NX ******\n");
            FakeObjectNonPagedPoolNxIoctlHandler = AllocateFakeObjectNonPagedPoolNxIoctlHandler(
                                                     Irp,
                                                     IrpSp: CurrentStackLocation);
            v7 = "****** HEVD_IOCTL_ALLOCATE_FAKE_OBJECT_NON_PAGED_POOL_NX ******\n";
            goto LABEL_62;
```

Example - HEVD vulnerable IOCTL

```
__int64 __fastcall TriggerBufferOverflowPagedPoolSession(void *UserBuffer, size_t Size)
{
  PVOID PoolWithTag; // rdi

  DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] Allocating Pool chunk\n");
  PoolWithTag = ExAllocatePoolWithTag(PoolType: PagedPoolSession, NumberOfBytes: 0x1F8u, Tag: 0x6B636148u);
  if ( PoolWithTag != nullptr )
  {
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] Pool Tag: %s\n", "'kcaH'");
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] Pool Type: %s\n", "PagedPoolSession");
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] Pool Size: 0x%X\n", 504);
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] Pool Chunk: 0x%p\n", PoolWithTag);
    ProbeForRead(Address: UserBuffer, Length: 0x1F8u, Alignment: 1u);
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] UserBuffer: 0x%p\n", UserBuffer);
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] UserBuffer Size: 0x%X\n", Size);
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] KernelBuffer: 0x%p\n", PoolWithTag);
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] KernelBuffer Size: 0x%X\n", 504);
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] Triggering Buffer Overflow in PagedPoolSession\n");
    memmove(a1: PoolWithTag, Src: UserBuffer, Size);
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] Freeing Pool chunk\n");
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] Pool Tag: %s\n", "'kcaH'");
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[+] Pool Chunk: 0x%p\n", PoolWithTag);
    ExFreePoolWithTag(P: PoolWithTag, Tag: 0x6B636148u);
    return 0;
  }
  else
  {
    DbgPrintEx(ComponentId: 0x4Du, Level: 3u, Format: "[-] Unable to allocate Pool chunk\n");
    return 3221225495LL;
  }
}
```

504-byte allocation

The variable Size is passed to memmove() with no validation against the allocated pool size.
This is a pool buffer overflow.

vulnerable line

Fix: check value of Size.

IDA 101

Open IDA > New > Select file to disassemble > analyze > load PDB file if available.

IDA should open in control flow graph view.

Space bar: switch between flat assembly / control flow graph view.

Note: IDA Free sends code to the cloud to decompile.

F5: generate decompiled C code.

Tab: switch between assembly / decompiled code.

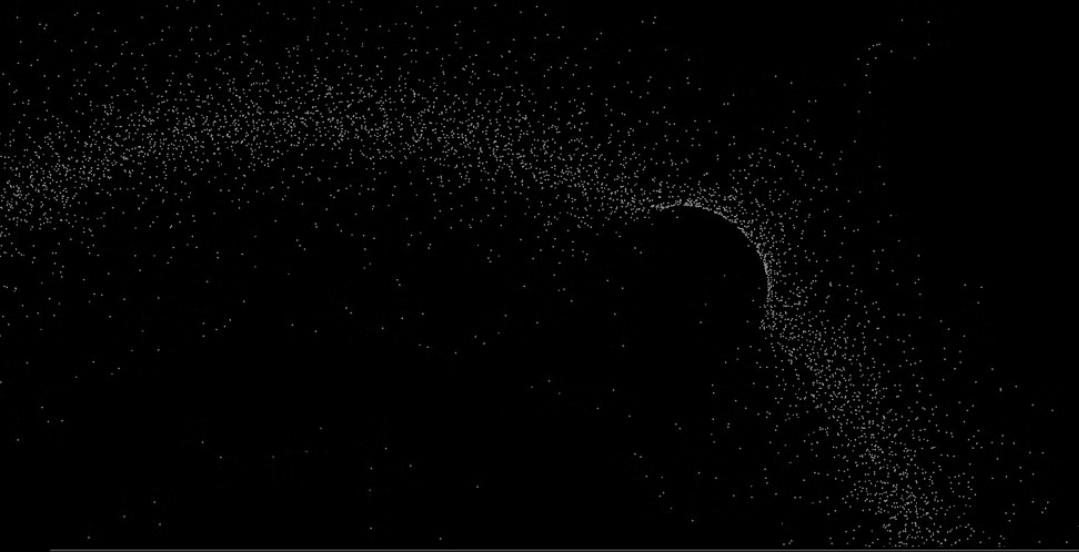
escape: back

g: go to a specific address.

x: show cross references to an address.

Vulnerability Types

General Vulnerability Types:
- Memory Corruption Bugs
- Concurrency Bugs
- Logic Bugs



Memory Corruption Bugs: Overflows

Overflow: data exceeds memory region.
Stack overflow (the classic bug): overwrite the stack to control the return address of a function.
Pool overflow: data written beyond pool boundaries (pool = kernel heap memory). (See 'Example - HEVD vulnerable IOCTL' slide).

Windows kernel drivers are more likely to contain heap overflows than stack overflows (kernel devs are taught to use the heap for large data structures).

Stack Overflow Example - CVE-2024-55407

```
26  switch ( LowPart )
27  {
28    case 0x80102040:
29      if ( (_DWORD)Options == 0 )
30        goto LABEL_30;
31      v9 = Options;
32      memmove(Dst: Src, Src: a2->AssociatedIrp.MasterIrp, MaxCount: Options);
33      v10 = sub_140001C0(BusAddress: Src[1], a2: &v17);
34      if ( v10 >= 0 )
35      {
36        memmove(Dst: p_Type, Src, MaxCount: v9);
37        a2->IoStatus.Information = v9;
38      }
39      a2->IoStatus.Status = v10;
40      break;
```

No validation check

WinIo64.sys

For IOCTLs 0x80102040 and 0x80102044, the destination buffer is PHYSICAL_ADDRESS[3] = 24 bytes. There's no validation check on the Options parameter - able to perform a kernel stack buffer overflow.

Memory Corruption Bugs: Use-After-Free

Use-after-free: object is allocated on the heap in user mode (or in kernel-mode pool memory) then is freed, but the code continues to use it.

Dangling pointer: pointer to a freed memory block is reused.

Common in big applications like browsers. Most common UAFs - from race conditions. Typical UAF occurs when a race happens and reference to an object is decremented to 0 then freed, a thread starts using the resource before it is freed.

Use-After-Free Example - HEVD

ExFreePoolWithTag(): deallocate block of memory

```
#ifdef SECURE
            //
            // Secure Note: This is secure because the developer is setting
            // 'g_UseAfterFreeObjectNonPagedPoolNx' to NULL once the Pool chunk is being freed
            //

            ExFreePoolWithTag((PVOID)g_UseAfterFreeObjectNonPagedPoolNx, (ULONG)POOL_TAG);

            //
            // Set to NULL to avoid dangling pointer
            //

            g_UseAfterFreeObjectNonPagedPoolNx = NULL;
#else
            //
            // Vulnerability Note: This is a vanilla Use After Free vulnerability
            // because the developer is not setting 'g_UseAfterFreeObjectNonPagedPoolNx' to NULL.
            // Hence, g_UseAfterFreeObjectNonPagedPoolNx still holds the reference to stale pointer
            // (dangling pointer)
            //

            ExFreePoolWithTag((PVOID)g_UseAfterFreeObjectNonPagedPoolNx, (ULONG)POOL_TAG);
#endif
```

Value is later set to NULL

Value is never set to NULL - chunk is available for another allocation to claim

HEVD Code

Memory Corruption Bugs: Double Free

Double free: calling free() on the same address twice without 2 calls to malloc(). Kernel level functions - ExAllocatePool()/ExFreePool(). Relies on how memory allocators use linked lists to manage freed memory. An attacker might be able to trick the allocator into returning pointers to arbitrary memory locations.

Understanding Double Free in Windows Kernel Drivers

Example user-mode code snippet:

```
char *p = malloc(64);  //allocating a pointer on the heap
free(p);               //freeing the pointer
//p isn't cleared, still points to the freed chunk that could be reused
free(p);  // double free — often exploitable
```

Concurrency Bugs

Race conditions: context switching can cause unexpected behavior when multiple threads access a shared resource. Program assumes a check is true when it is performing an action.

TOCTOU: time-of-check to time-of-use. Time of check: process inspects a condition or state. Time of use: process acts upon that assumption. TOCTOU bugs are common in Windows drivers & can be found with static analysis, other concurrency bugs require dynamic analysis.

An attacker can try to modify the condition / state before a variable is used.

Logic Bugs

Reference counting: kernel object tracking. Each heap object keeps count of the number of references to that object.

Might be able to cause a use-after-free: attacker can manipulate an object so that its reference count is decremented prematurely, causing the program to use a freed region of memory.

Type confusion: object is allocated as a certain type and then is referenced as a different type. **Might let an attacker trigger memory corruption.**

Other logic bugs: functionality a program shouldn't have!

Logic Bug Example - capcom.sys

capcom.sys - Was an anti-cheat driver for Street Fighter V.

An IOCTL disables SMEP (Windows memory protection) then takes in and executes a user supplied function pointer.

```
__int64 __fastcall sub_10524(void (__fastcall *a1)(_QWORD))
{
  __int64 v2; // [rsp+20h] [rbp-28h] BYREF
  void (__fastcall *v3)(PVOID (__stdcall *)(PUNICODE_STRING)); // [rsp+28h] [rbp-20h]
  PVOID (__stdcall *v4)(PUNICODE_STRING); // [rsp+30h] [rbp-18h]

  if ( *((void (__fastcall **)(_QWORD))a1 - 1) != a1 )
    return 0;
  v3 = (void (__fastcall *)(PVOID (__stdcall *)(PUNICODE_STRING)))a1;
  v4 = MmGetSystemRoutineAddress;
  v2 = 0;
  sub_10788(a1: &v2);
  v3(a1: v4);
  sub_107A0(a1: &v2);
  return 1;
}
```

executing a user-supplied pointer

reenabling SMEP using compiler intrinsic

disabling SMEP using compiler intrinsic

```
unsigned __int64 __fastcall sub_10788(unsigned __int64 *a1)
{
  unsigned __int64 v1; // rax
  unsigned __int64 result; // rax

  _disable();
  v1 = __readcr4();
  *a1 = v1;
  result = v1 & 0xFFFFFFFFFFEFFFFFuLL;
  __writecr4(result);
  return result;
}
```

Access Tokens and Privilege Escalation

Common attacker technique: replace access token in a process with the elevated access token of a SYSTEM process. Requires a write primitive to overwrite the access token in the EPROCESS structure.

Privilege escalation technique.

Exploit Primitives

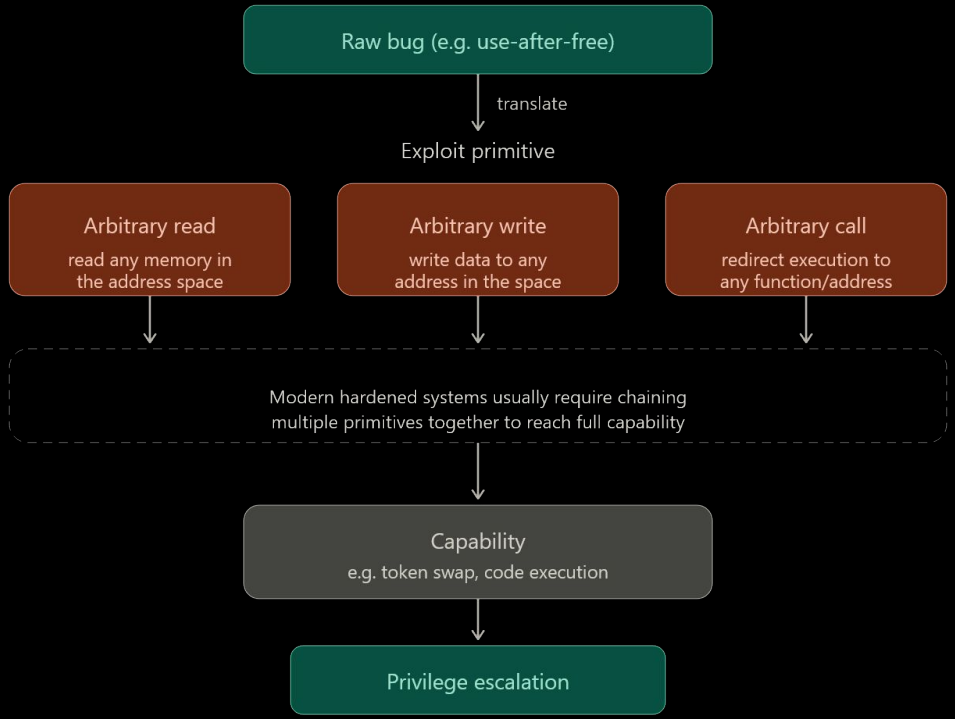
Primitive: translating a raw bug (like use-after-free) into an actual capability.

3 main categories of primitives:

- **arbitrary read**: read from any memory in the process address space.
- **arbitrary write**: write arbitrary data to an arbitrary location in the process address space.
- **arbitrary call**: redirect execution flow to any function or address in the process address space.

Modern hardened systems: need to chain together multiple primitives.

Exploit Primitives



Kernel Exploitation

Kernel exploitation: Harder with every version of Windows.
Goal: escalate privileges, stealth.
How: run code in the kernel to read and write from model specific registers (MSRs), steal privileged access tokens, patch kernel structs. (MSR - special registers, used to config CPU).
To load drivers into a modern Windows Kernel you either need a valid certificate, a n/zero-day exploit or to bring your vulnerable driver (BYOVD).

General Exploit Steps

- Find a bug
- Trigger a crash
- Control the data you can send
- Build an exploit primitive
- Escalate privileges

Driver Vulnerability Hunting

- Driver Identification
- Driver Static Analysis
- Driver Dynamic Analysis

We will only be doing driver identification and static analysis in this class.

Driver Identification

- Look at driver metadata
- Look at strings present in the driver
- Identify the driver framework (WDM vs WDF)
- Look for any known vulnerabilities

Driver Static Analysis

- Load driver into reversing tool
- Identify the entry point (DriverEntry)
- Find the device name and symbolic link
- Identify the device control function(s) (look for switch cases or if/else statements)
- Identify IOCTL numbers and arguments to call them
- Identify privileges needed to call IOCTLs
- Identify functions import and driver capabilities

Dynamic Driver Analysis

- Install driver on a target VM
- Method to call IOCTLs (tools available)
- Step through execution using a kernel debugger
- Set a breakpoint on the IOCTL handler, step through execution
- Look at kernel memory, objects and state
- Unload driver when done

Interesting Kernel-Mode Functions

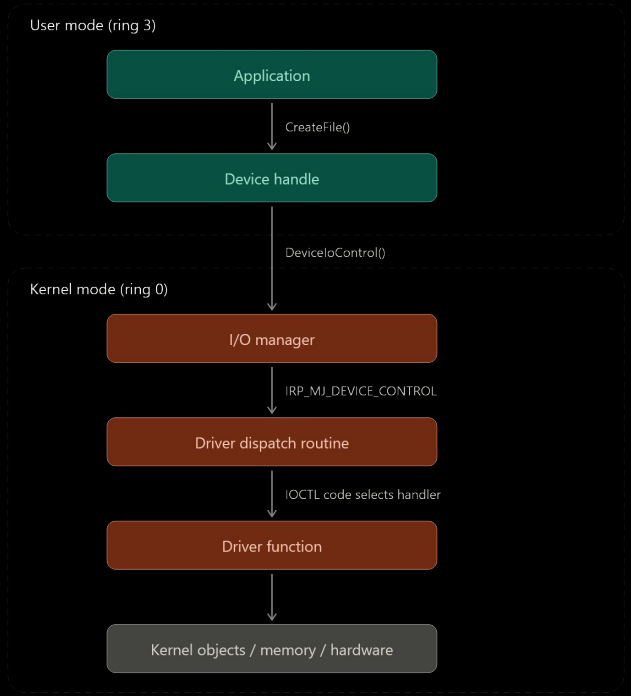
(Look for standard Windows functions in the Imports tab. C runtime routines and compiler intrinsics will not show up there.)

Does a driver use functions to:

- copy memory? memcpy(), memmove(), MmCopyMemory(), MmCopyVirtualMemory(), RtlCopyMemory(), RtlMoveMemory()
- map physical memory? MmMapIoSpace(), MmMapIoSpaceEx(), ZwMapViewOfSection()
- allocate kernel memory? ExAllocatePool*()
- open a process? ZwOpenProcess()
- access MSRs? __readmsr, __writemsr compiler intrinsics

There are plenty more things to look for but we will focus on these for this class!

Overview


User mode (ring 3)
Application
CreateFile()
Device handle
DeviceIoControl()
Kernel mode (ring 0)
I/O manager
IRP_MJ_DEVICE_CONTROL
Driver dispatch routine
IOCTL code selects handler
Driver function
Kernel objects / memory / hardware

Exercise - Analyzing a Vulnerable Driver

Perform driver identification.

Perform static analysis on the driver.

Use the next slides for hints / walkthrough or try it on your own!

RTCore64.sys - click "Download Vulnerable Sample" - keep this as a .bin file unless you are on an isolated VM system!

Driver Identification Steps

- Load the driver in IDA
- Identify what framework the driver uses: WDM, KMDF or UMDF (WDM drivers initialize the driver object using IoCreateDevice())
- Note interesting functions the driver imports
- Note interesting functions not imported (C runtime library)
- Note any interesting strings

DriverEntry Static Analysis

- Find DriverEntry
- Identify the created device
- Identify the symbolic link for the device
- Identify the MajorFunction table
- Locate the IOCTL handler

IOCTL Handler Static Analysis

- Identify IOCTL numbers
- Identify any possible vulnerable functions called by an IOCTL (or research the driver to find any known vulnerabilities)
- Identify the input structure needed for the IOCTL
- Are any interesting functions used?
- Can you provide any input to those functions?

CVE-2019-16098

Driver designed to tune GPU hardware, used in multiple ransomware campaigns.

CVE-2019-16098 PoC

Analysis Writeup

Used in a demo to nullify EDR capabilities at CYBERSEC 2023

Usage by Ransomware Groups

Known Vulnerabilities - CVE-2019-16098

Multiple vulnerabilities under the CVE.

Any logged-in local user can send commands to the driver without any restrictions (bad practice).

Arbitrary memory read and write primitives.

Arbitrary read and write to MSRs.

IOCTL requests without any access control allow an attacker to:

- get a read and a write primitive.
- escalate privileges to kernel level.
- overwrite an access token and escalate privileges.

Finding the entry point

Load driver in IDA, find DriverEntry (functions tab)

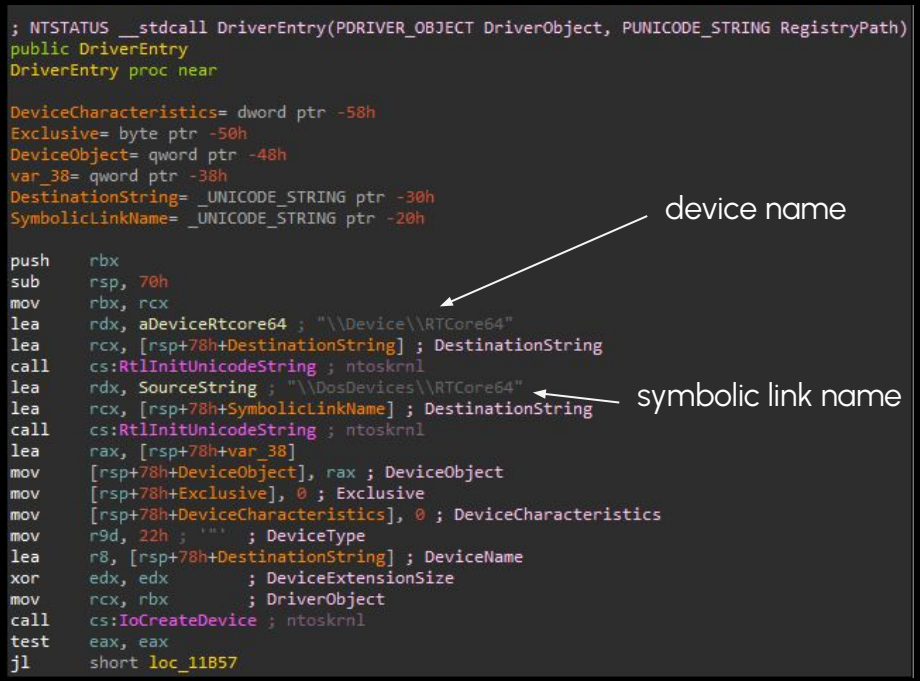
```
; NTSTATUS __stdcall DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
public DriverEntry
DriverEntry proc near

DeviceCharacteristics= dword ptr -58h
Exclusive= byte ptr -50h
DeviceObject= qword ptr -48h
var_38= qword ptr -38h
DestinationString= _UNICODE_STRING ptr -30h
SymbolicLinkName= _UNICODE_STRING ptr -20h
```

Generating decompiled "pseudocode" in IDA: click into assembly block, then press F5. Use Tab to switch between asm / C.

Note: IDA's decompiled code is pretty accurate but it's always best to verify it against the assembly!

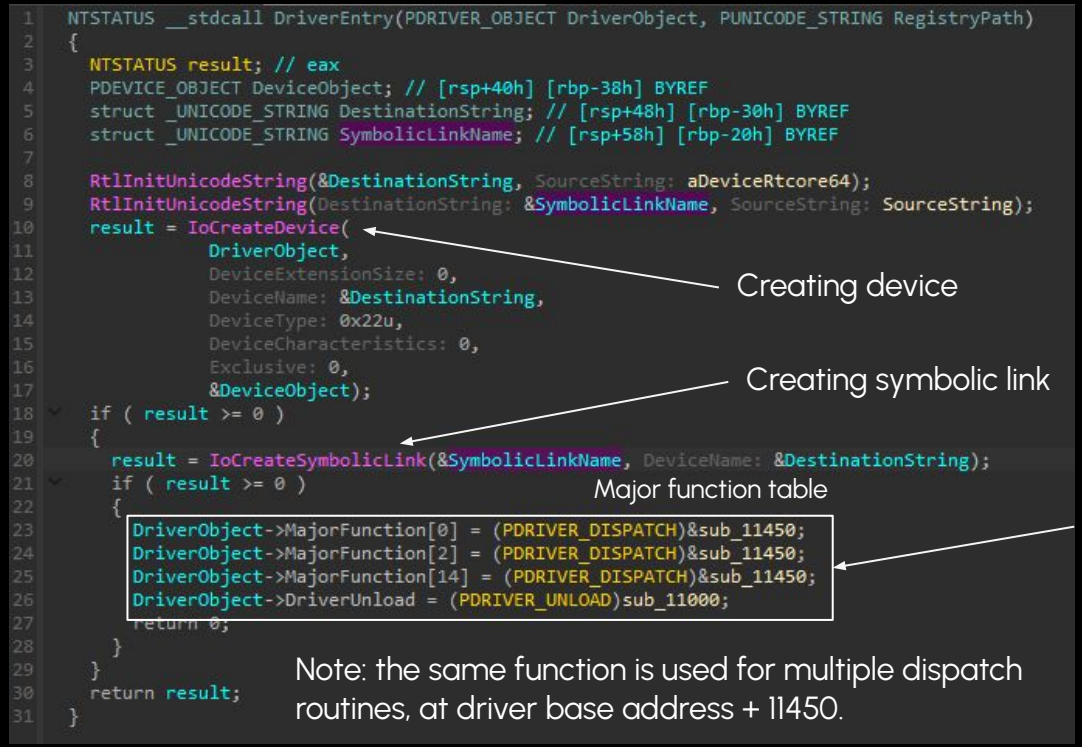
Identifying the Device and Symbolic Link



IoCreateDevice() - means this is a WDM driver.

An attacker would use "\\\\.\\RTCore64" to open a handle to the device.

DriverEntry Decompiled Code



DriverEntry Notes

MajorFunction table: correlates different IRP codes to functions in the driver.

Registering function callbacks for each IRP code in the MajorFunction table.

0 - IRP_MJ_CREATE - opens a handle to the device.

2 - IRP_MJ_CLOSE - closes a handle to the device.

14 - IRP_MJ_DEVICE_CONTROL - processes IOCTL requests. We will need to check this function out to identify IOCTLs!

What is a callback? -> setting a pointer value to a function that will be called when a specific action occurs.

Kernel callbacks: notify a Windows Kernel Driver when a specific event occurs.

CVE-2019-16098 - IOCTL Handler

```
__int64 __fastcall sub_11450(__int64 a1, IRP *a2)
{
  struct _IO_STACK_LOCATION *CurrentStackLocation; // rax
  int *p_Type; // r10
  __int64 Options; // rdx
  __int64 v6; // r9
  __int64 v7; // rcx
  __int64 v8; // rdx
  int v9; // eax
  void *v10; // rcx
  int v11; // eax
  char v12; // al
  unsigned __int16 v13; // ax
  unsigned __int32 v14; // eax
  __int64 v15; // r10
  unsigned __int16 *v16; // r10
  __int64 v17; // r10
  int v18; // ecx
  ULONG Length; // ecx
  ULONG v20; // ecx
  ULONG Offset; // edx
  unsigned int Status; // ebx
  unsigned __int64 v24; // [rsp+30h] [rbp-28h]

  a2->IoStatus.Status = 0;
  a2->IoStatus.Information = 0;
  CurrentStackLocation = a2->Tail.Overlay.CurrentStackLocation;
  p_Type = (int *)&a2->AssociatedIrp.MasterIrp->Type;
  Options = CurrentStackLocation->Parameters.Create.Options;
  v6 = CurrentStackLocation->Parameters.Read.Length;
  if ( CurrentStackLocation->MajorFunction == 14 )
  {
    switch ( CurrentStackLocation->Parameters.Read.ByteOffset.LowPart )
```

Switch on IOCTL

CVE-2019-16098 - IOCTL 0x80002048

```
249         case 0x80002048:
250           if ( (_DWORD)Options == 48 )
251           {
252             v7 = MasterIrp->MdlAddress;
253             if ( v7 != nullptr )
254             {
255               switch ( MasterIrp->AssociatedIrp.IrpCount )
256               {
257                 case 1:
258                   HIDWORD(MasterIrp->AssociatedIrp.SystemBuffer) = *((unsigned __int8 *)&v7->Next
259                                                                    + *(&MasterIrp->Flags + 1));
260                   break;
261                 case 2:
262                   HIDWORD(MasterIrp->AssociatedIrp.SystemBuffer) = *(unsigned __int16 *)((char *)&v7->Next
263                                                                                        + *(&MasterIrp->Flags + 1));
264                   break;
265                 case 4:
266                   HIDWORD(MasterIrp->AssociatedIrp.SystemBuffer) = *(_DWORD *)((char *)&v7->Next + *(&MasterIrp->Flags + 1));
267                   break;
268                 default:
269                   break;
270               }
271               a2->IoStatus.Status = 0;
272               a2->IoStatus.Information = 48;
273             }
274             else
275             {
276               a2->IoStatus.Status = -1073741811;
277             }
278           }
```

Arbitrary 4-byte memory read.

Reads contents from a user-supplied virtual address into SystemBuffer.

CVE-2019-16098 - IOCTL 0x8000204C

```
284         case 0x8000204C:
285           if ( (_DWORD)Options == 48 )
286           {
287             v8 = MasterIrp->MdlAddress;
288             if ( v8 != nullptr )
289             {
290               switch ( MasterIrp->AssociatedIrp.IrpCount )
291               {
292                 case 1:
293                   *((_BYTE *)&v8->Next + *(&MasterIrp->Flags + 1)) = BYTE4(MasterIrp->AssociatedIrp.SystemBuffer);
294                   break;
295                 case 2:
296                   *(_WORD *)((char *)&v8->Next + *(&MasterIrp->Flags + 1)) = WORD2(MasterIrp->AssociatedIrp.SystemBuffer);
297                   break;
298                 case 4:
299                   *(_DWORD *)((char *)&v8->Next + *(&MasterIrp->Flags + 1)) = HIDWORD(MasterIrp->AssociatedIrp.SystemBuffer);
300                   break;
301                 default:
302                   break;
303               }
304               a2->IoStatus.Status = 0;
305               a2->IoStatus.Information = 48;
306             }
307             else
308             {
309               a2->IoStatus.Status = 3221225626;
310             }
311           }
312           else
313           {
314             a2->IoStatus.Status = -1073741811;
315           }
316           break;
```

Arbitrary 4-byte memory write.

Writes from a user-determined virtual address into SystemBuffer.

CVE-2019-16098 - IOCTL 0x80002040

```
215         case 0x80002040:
216           if ( (_DWORD)Options == 48 )
217           {
218             v9 = sub_113A0(a1: MasterIrp, a2: Options, a3: &loc_114B6, a4: v6);
219             a2->IoStatus.Status = v9;
220             if ( v9 < 0 )
221               a2->IoStatus.Status = -1073741670;
222             else
223               a2->IoStatus.Information = 48;
224           }
225           else
226           {
227             a2->IoStatus.Status = -1073741811;
228           }
229           break;
```

Not vulnerable but can be used alongside the memory read / write primitives to modify hardware firmware data.

Maps physical memory.

User supplies physical address and length, returns a kernel virtual address that can be used with IOCTLs.

```
1   __int64 __fastcall sub_113A0(PHYSICAL_ADDRESS *a1)
2   {
3     PVOID v3; // rax
4     ULONG AddressSpace; // [rsp+30h] [rbp-18h] BYREF
5     LARGE_INTEGER PhysicalAddress; // [rsp+38h] [rbp-10h] BYREF
6
7     if ( (unsigned __int8)sub_11060(a1: a1->QuadPart, a2: a1[2].LowPart, a3: &a1[4]) == 0 )
8       return 3221225473LL;
9     AddressSpace = 0;
10    if ( HalTranslateBusAddress(
11           InterfaceType: Isa,
12           BusNumber: 0,
13           BusAddress: *a1,
14           &AddressSpace,
15           TranslatedAddress: &PhysicalAddress) == 0 )
16      return 3221225626LL;
17    v3 = MmMapIoSpace(PhysicalAddress, NumberOfBytes: a1[2].LowPart, CacheType: MmNonCached);
18    if ( v3 == nullptr )
19      return 3221225626LL;
20    a1[1].QuadPart = (LONGLONG)v3;
21    return 0;
22  }
```

CVE-2019-16098

Arbitrary memory read – IOCTL 0x80002048.

Arbitrary memory write – IOCTL 0x8000204C.

Both due to no validation on a user-determined virtual address.

An attacker can read from a privileged processes access token in EPROCESS and overwrite the access token of another process (4 bytes at a time, addresses in nonpaged kernel virtual memory).

To modify hardware settings: use IOCTL 0x80002040 to map a physical address into virtual kernel space then use the vulnerable IOCTLs (0x80002048 or 0x8000204C) to read or write to it.

CVE-2019-16098

Arbitrary MSR read – IOCTL 0x80002030.

Arbitrary MSR write – IOCTL 0x80002034.

Read / write MSRs - modify CPU settings. Getting the memory address of the operating system's system call handler can let an attacker bypass KASLR.

Understanding Arbitrary Access Primitives in Windows Kernel

CVE-2019-16098 - arbitrary MSR read

Arbitrary MSR read – IOCTL 0x80002030

```
189         case 0x80002030:
190           if ( (_DWORD)Options == 12 )
191           {
192             v24 = __readmsr(*p_Type);
193             p_Type[1] = HIDWORD(v24);
194             p_Type[2] = v24;
195             a2->IoStatus.Status = 0;
196             a2->IoStatus.Information = 12;
197           }
198           else
199           {
200             a2->IoStatus.Status = -1073741811;
201           }
202           break;
```

user supplies p_Type value - controls what MSR to read

Can leverage MSR to leak base of the kernel (ntoskernl.exe) - then use the physical write primitive to change kernel addresses.

CVE-2019-16098 - arbitrary MSR write

Arbitrary MSR write – IOCTL 0x80002034

Might be able to overwrite syscall entry pointer and control execution.

```
203         case 0x80002034:
204           if ( (_DWORD)Options == 12 )
205           {
206             __writemsr(*p_Type, __PAIR64__(p_Type[1], p_Type[2]));
207             a2->IoStatus.Status = 0;
208             a2->IoStatus.Information = 12;
209           }
210           else
211           {
212             a2->IoStatus.Status = -1073741811;
213           }
214           break;
```

Bonus exercise!

If previous exercise is completed: pick a vulnerable driver from loldrivers and perform the same static analysis steps.

Research for any relevant CVE / write ups details and compare your analysis.

Discussion

What can you do with the memory read / write primitives from the driver?

What could you do with the MSR read / write primitives?

Any ideas of how OS protections could stop these actions?

Discussion

What can you do with the read / write primitives from the driver? Modify kernel space virtual memory (including key kernel structures), execute shellcode, neutralize mitigations, create an unstable system leading to a BSOD.

How would the different OS protections affect this? There's a variety of different protections in Windows - to detect tampering on key kernel data structures, make it harder to guess memory addresses, or execute shellcode.

Discussion - memory read / write primitive

Windows mitigations: prevent you from getting a primitive, prevent code execution after getting a primitive, detects tampering.

None of these mitigations prevent data tampering!

Classic approach is the best bet: locate SYSTEM access token and use it to elevate privileges by writing it to the access token in EPROCESS of a process you control (read to get the token, then write it. No shellcode executes!)

Discussion - MSR read / write primitive

MSRs: CPU specific registers.

MSR Read: get address of registers like IA32_LSTAR (x86-64 syscall entry point) or IA32_SYSENTER_EIP (x86 syscall entry point) > bypass kernel memory protections (kASLR) by calculating the base of ntoskrnl.exe with a known offset.

MSR write: write to those registers and control execution flow, hijacking the syscall entry point (HVCI in modern Windows mitigates this).

Understanding Arbitrary Access Primitives in Windows Kernel

Learn More - Books

Windows Internals - Pavel Yosifovich, Alex Ionescu, Mark E. Russinovich, and David A. Solomon

Windows Kernel Programming - Pavel Yosifovich

A Guide to Kernel Exploitation - Enrico Perla and Massimiliano Oldani

The Rootkit Arsenal - Bill Blunden

Practical Reverse Engineering - Alexandre Gazet, Bruce Dang, and Elias Bachaalany

Evading EDR - Matt Hand

Learn More - Dev Resources

- Windows Driver Docs
- Windows Driver Security Guidance
- OS Dev Wiki
- OSR Dev Blog
- NtDocs
- Vergilius Project
- React OS

Learn More - Blogs

- Abatchy Tutorials
- Fuzzy Security Tutorials
- OSR Online - Legacy
- OSR Dev Blog
- Codemachine Articles
- Unknown Cheats
- Connor McGarr
- Alex Ionescu
- POPKORN Paper

Learn More - Practice

Open Security Training

Windows Kernel Exploits

HackSysExtremeVulnerableDriver

Windows Exploitation Resources

Pwncollege - Advanced Software Exploitation

Thank you

